kubenetmon

How we built a tool to measure data transfer in ClickHouse Cloud

Ilya Andreev, Engineering @ ClickHouse

ClickHouse

Agenda

01 Defining our network observability needs

02 Evaluating existing tools for our purpose

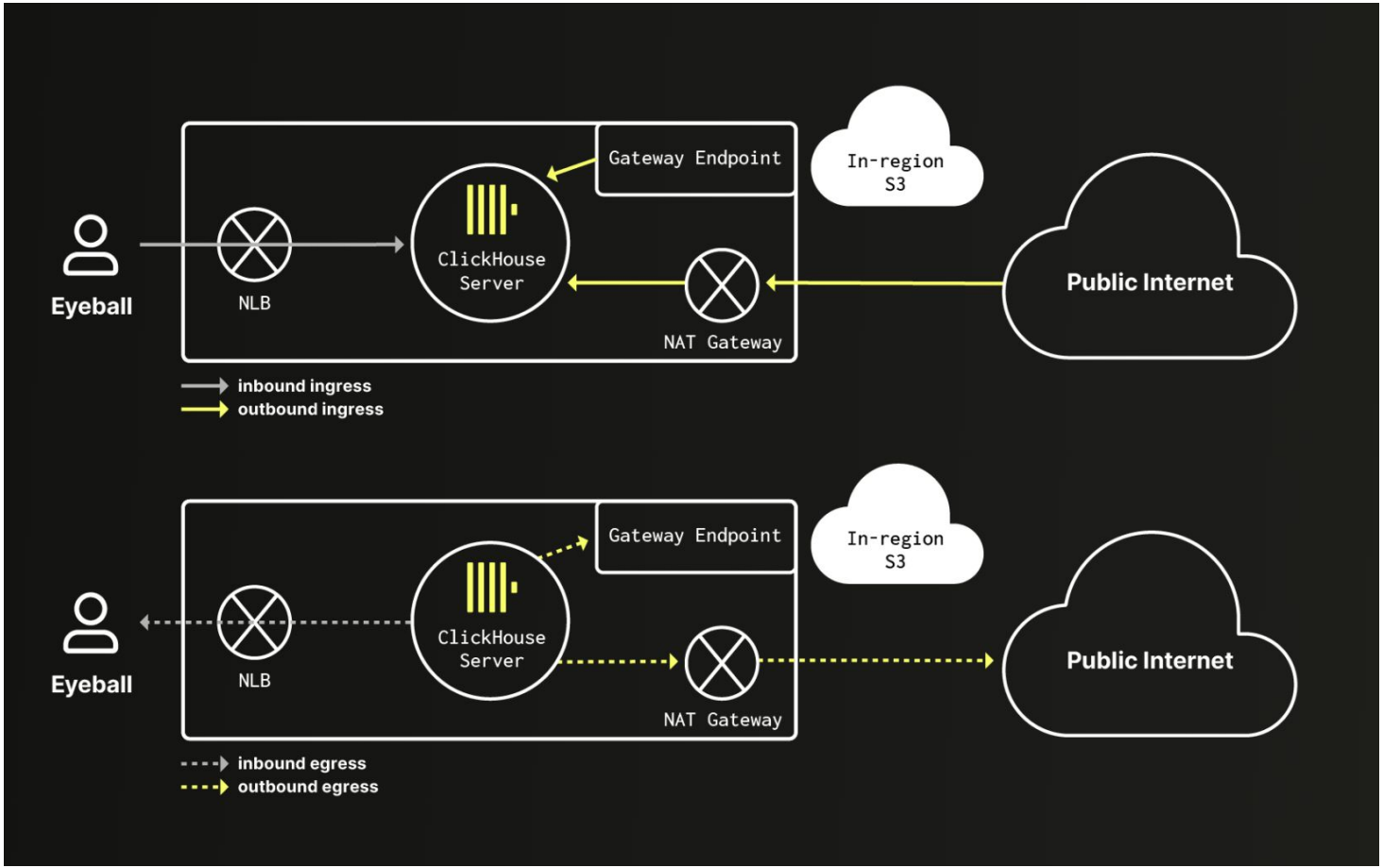
03 Building and open sourcing kubenetmon

©2025 ClickHouse, Inc. All Rights Reserved



01 Defining our network observability needs

ClickHouse



©2025 ClickHouse, Inc. All Rights Reserved

High-level requirements

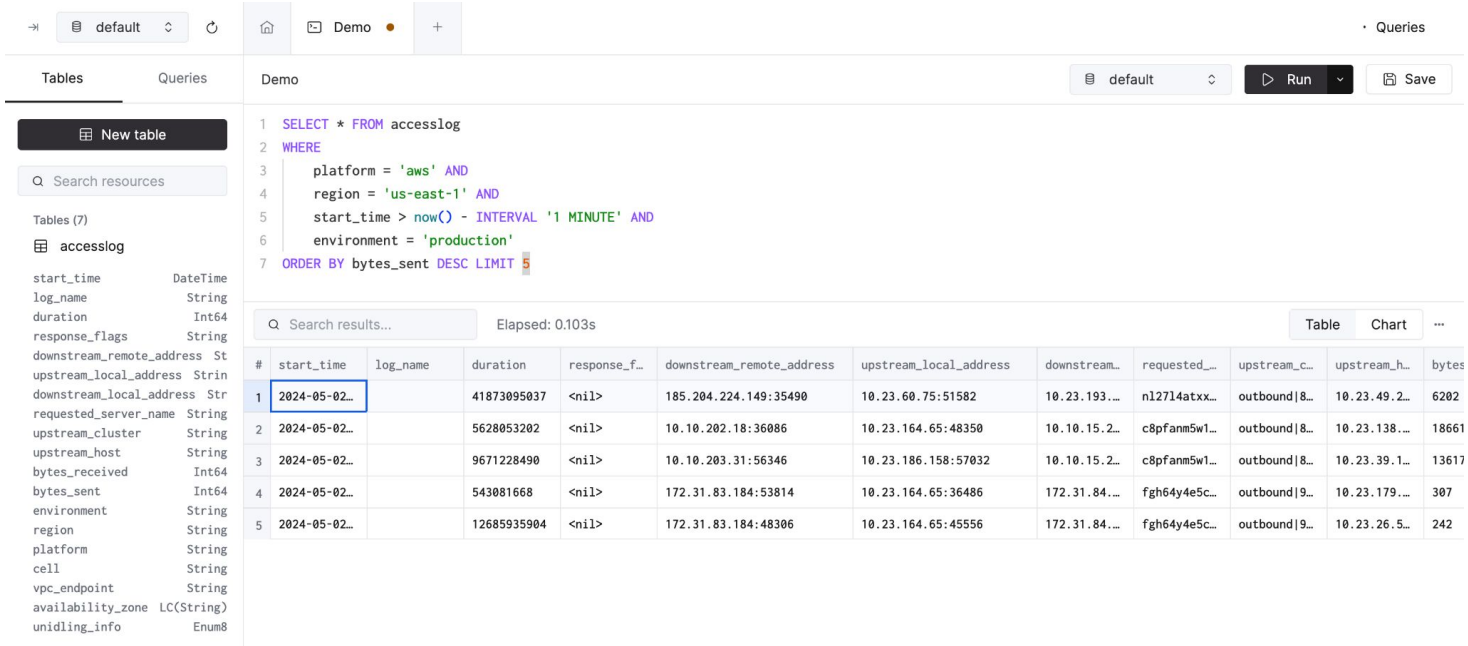
- **Forensics**
  - Limit cold trails on incidents and customer escalations
- **Metering**
  - What is this customer’s usage?
- **Accounting**
  - How much are we spending on specific use-cases?

©2025 ClickHouse, Inc. All Rights Reserved



Network, what do we know about it?

Connection records from our proxy



©2025 ClickHouse, Inc. All Rights Reserved

Network, what do we know about it?

Tons of data available in ClickHouse system tables!

- query_log.ProfileEvents
  - S3ReadBytes, S3WriteBytes
  - S3ReadRequestsCount, S3WriteRequestsCount
  - S3ReadMicroseconds, S3WriteMicroseconds
  - NetworkReceiveBytes, NetworkSendBytes
- metrics
  - TCPConnection
- and other things...

©2025 ClickHouse, Inc. All Rights Reserved



Network, what do we know about it?

- node-exporter metrics
  - node_network_receive_bytes_total and the like
- Cloud Provider flow logs

©2025 ClickHouse, Inc. All Rights Reserved



More concrete requirements

- **Forensics**
  - Clear reference point to observe from
  - Connections, not just business categories, e.g. “TCP 10.23.24.24:60123 → 8.8.8.8:443” instead of “cross-region egress”
  - Bytes and packets, not just $
  - Rates, not just grand aggregates, e.g. “1.1 MBps” instead “5 TB per month”
- **Metering**
  - Attribution and ownership, i.e. pod names, instance names, sampling points
- **Accounting**
  - Roll up the above by categories and periods

©2025 ClickHouse, Inc. All Rights Reserved



02 Evaluating existing tools for our purpose

ClickHouse

Available options

- Proxy accesslog
  - Blind spots, connection-oriented
- ClickHouse ProfileEvents and metrics
  - Query-oriented
- VPC Flow Logs
  - Not always Kubernetes-aware, different across providers, unexpected limitations
- Cost metering tools
  - Inflexible, missing forensics
- Cilium and Hubble
  - No-context volumetric data vs context-rich non-volumetric data

©2025 ClickHouse, Inc. All Rights Reserved





Our solution

ClickHouse

How to collect connection data?

Whether running with kube-proxy replacement or not, standard Cilium host routing keeps conntrack available to us:

```
# conntrack -L
tcp      6 35 TIME_WAIT src=10.1.1.42 dst=10.1.2.228 sport=41442 dport=5555
src=10.1.2.228 dst=10.1.1.42 sport=5555 dport=41442 [ASSURED] mark=0 use=1
tcp      6 431999 ESTABLISHED src=10.1.0.27 dst=10.1.131.55 sport=37670
dport=10249 src=10.1.131.55 dst=10.1.0.27 sport=10249 dport=37670 [ASSURED]
mark=0 use=1
```

©2025 ClickHouse, Inc. All Rights Reserved



What about packets and bytes?

Check typical sizes of connection tables and the kernel path for incrementing conntrack counters, and then enable them:

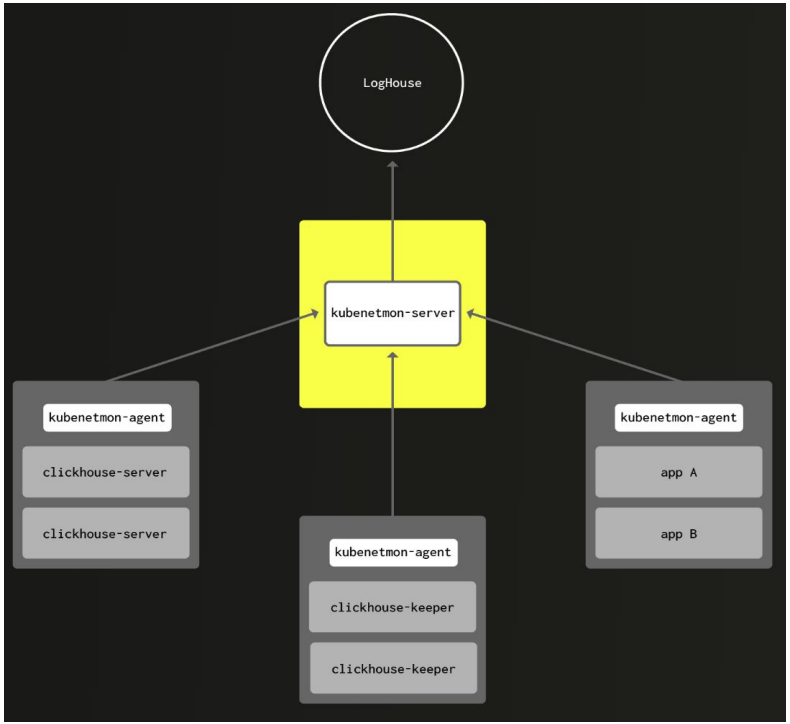
```
# echo "1" > /proc/sys/net/netfilter/nf_conntrack_acct
# conntrack -L
tcp      6 35 TIME_WAIT src=10.1.1.42 dst=10.1.2.228 sport=41442 dport=5555
packets=334368 bytes=106591692 src=10.1.2.228 dst=10.1.1.42 sport=5555
dport=41442 packets=280371 bytes=692287110 [ASSURED] mark=0 use=1
tcp      6 431999 ESTABLISHED src=10.1.0.27 dst=10.1.131.55 sport=37670
dport=10249 packets=20 bytes=21564 src=10.1.131.55 dst=10.1.0.27
sport=10249 dport=37670 packets=11 bytes=2011 [ASSURED] mark=0 use=1
```

©2025 ClickHouse, Inc. All Rights Reserved



Attributing connections to endpoints

- Polling Kubernetes API vs using Informers
- Hub-and-spoke vs flat topology



©2025 ClickHouse, Inc. All Rights Reserved

Storing connection data

- Columnar database: add as many columns as you want
- SummingMergeTree deals with scraping jitter for us

```
CREATE TABLE default.network_flows_0
(
    `date` Date CODEC(Delta(2), ZSTD(1)),
    `intervalStartTime` DateTime CODEC(Delta(4), ZSTD(1)),
    `intervalSeconds` UInt16 CODEC(Delta(2), ZSTD(1)),
    `environment` LowCardinality(String) CODEC(ZSTD(1)),
    `proto` LowCardinality(String) CODEC(ZSTD(1)),
    `connectionClass` LowCardinality(String) CODEC(ZSTD(1)),
    `connectionFlags` Map(LowCardinality(String), Bool) CODEC(ZSTD(1)),
    `direction` Enum('out' = 1, 'in' = 2) CODEC(ZSTD(1)),
    `localCloud` LowCardinality(String) CODEC(ZSTD(1)),
    `localRegion` LowCardinality(String) CODEC(ZSTD(1)),
    `localCluster` LowCardinality(String) CODEC(ZSTD(1)),
    `localCell` LowCardinality(String) CODEC(ZSTD(1)),
    `localAvailabilityZone` LowCardinality(String) CODEC(ZSTD(1)),
    `localNode` String CODEC(ZSTD(1)),
    `localInstanceID` String CODEC(ZSTD(1)),
    `localNamespace` LowCardinality(String) CODEC(ZSTD(1)),
    `localPod` String CODEC(ZSTD(1)),
    `localIPv4` IPv4 CODEC(Delta(4), ZSTD(1)),
    `localPort` UInt16 CODEC(Delta(2), ZSTD(1)),
    `localApp` String CODEC(ZSTD(1)),
    `remoteCloud` LowCardinality(String) CODEC(ZSTD(1)),
    `remoteRegion` LowCardinality(String) CODEC(ZSTD(1)),
    `remoteCluster` LowCardinality(String) CODEC(ZSTD(1)),
    `remoteCell` LowCardinality(String) CODEC(ZSTD(1)),
    `remoteAvailabilityZone` LowCardinality(String) CODEC(ZSTD(1)),
    `remoteNode` String CODEC(ZSTD(1)),
    `remoteInstanceID` String CODEC(ZSTD(1)),
    `remoteNamespace` LowCardinality(String) CODEC(ZSTD(1)),
    `remotePod` String CODEC(ZSTD(1)),
    `remoteIPv4` IPv4 CODEC(Delta(4), ZSTD(1)),
    `remotePort` UInt16 CODEC(Delta(2), ZSTD(1)),
    `remoteApp` String CODEC(ZSTD(1)),
    `remoteCloudService` LowCardinality(String) CODEC(ZSTD(1)),
    `bytes` UInt64 CODEC(Delta(8), ZSTD(1)),
    `packets` UInt64 CODEC(Delta(8), ZSTD(1))
)
ENGINE = SummingMergeTree((bytes, packets))
PARTITION BY date
PRIMARY KEY (date, intervalStartTime, direction, proto, localApp,
remoteApp, localPod, remotePod)
ORDER BY (date, intervalStartTime, direction, proto, localApp, remoteApp,
localPod, remotePod, intervalSeconds, environment, connectionClass,
connectionFlags, localCloud, localRegion, localCluster, localCell,
localAvailabilityZone, localNode, localInstanceID, localNamespace,
localIPv4, localPort, remoteCloud, remoteRegion, remoteCluster, remoteCell,
remoteAvailabilityZone, remoteNode, remoteInstanceID, remoteNamespace,
remoteIPv4, remotePort, remoteCloudService)
TTL intervalStartTime + toIntervalDay(90)
SETTINGS index_granularity = 8192, ttl_only_drop_parts = 1
```

©2025 ClickHouse, Inc. All Rights Reserved



Don’t have to worry about storage

```
SELECT
    formatReadableQuantity(sum(rows)),
    formatReadableSize(sum(data_compressed_bytes)) AS compressed_size,
    formatReadableSize(sum(data_uncompressed_bytes)) AS uncompressed_size,
    sum(data_compressed_bytes) / sum(data_uncompressed_bytes) AS
compression_ratio
FROM system.parts
WHERE (database = 'default') AND (`table` = 'network_flows_0') AND (active
= 1);
```

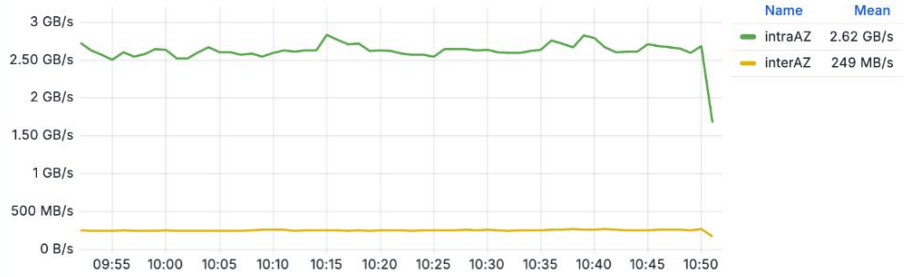
```
Row 1:
──────
formatReadab⋯(sum(rows)): 399.05 billion
compressed_size:          2.66 TiB
uncompressed_size:        86.69 TiB
compression_ratio:        0.030629224420511452
1 row in set. Elapsed: 0.116 sec.
```

©2025 ClickHouse, Inc. All Rights Reserved

Insights served

Cluster-local East-West (ignoring node-local)

Cross-AZ traffic



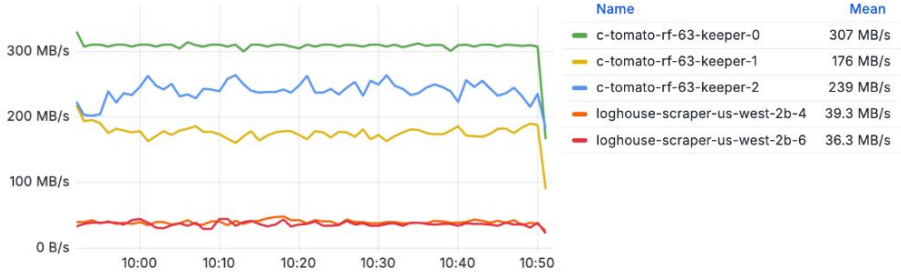
Top localPod talkers by throughput

localPod	bytes	packets
c-tomato-rf-63-keeper-0	1.10 TB	50.8 Mil
c-tomato-rf-63-keeper-2	856 GB	52.4 Mil
c-tomato-rf-63-keeper-1	630 GB	37.3 Mil
loghouse-scraper-us-west-2b-4	141 GB	9.06 Mil
loghouse-scraper-us-west-2b-6	130 GB	8.05 Mil
loghouse-scraper-us-west-2c-2	119 GB	7.67 Mil

Top (localPod, remotePod) inter-AZ talkers by throughput

localPod	remotePod	bytes	packets
c-tomato-rf-63-keeper-2	c-tomato-rf-63-server-5qv5	349 GB	18.5 Mil
c-tomato-rf-63-keeper-0	c-tomato-rf-63-server-n64q	338 GB	13.5 Mil
c-tomato-rf-63-keeper-0	c-tomato-rf-63-server-u5mj	315 GB	13.2 Mil
c-tomato-rf-63-keeper-2	c-maroon-oe-40-server-g62	281 GB	17.9 Mil
c-tomato-rf-63-keeper-1	c-tomato-rf-63-server-u9c0	275 GB	14.6 Mil
c-tomato-rf-63-keeper-2	c-lime-mi-60-server-8cw2xa	229 GB	14.8 Mil

Bytes per second for top 5 localPod talkers by throughput



©2025 ClickHouse, Inc. All Rights Reserved

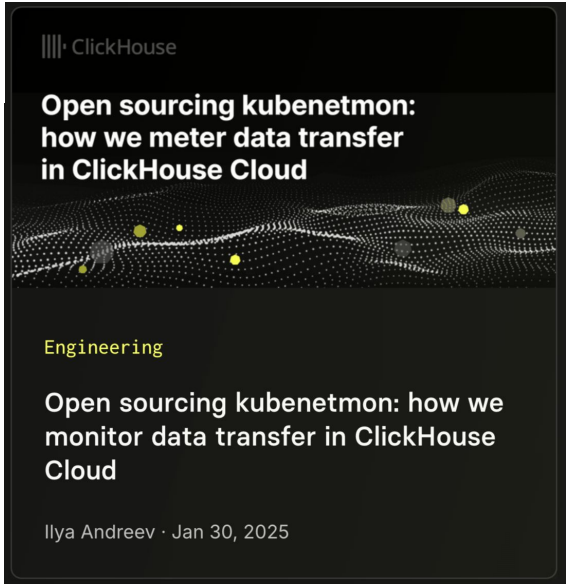
kubenetmon

go report A+ | Lint and test charts and code passing | License Apache 2.0 | release kubenetmon-server-1.0.2

📣💛 News

> **Blog:** Read kubenetmon announcement blogpost and learn how it was built: https://clickhouse.com/blog/kubenetmon-open-sourced!

kubenetmon GitHub



©2025 ClickHouse, Inc. All Rights Reserved

Thank you!

